HW3 Submission: Context Engineering, Rules & Process

Student: Haomiao Shi · **Course:** CS6983 AI Coding · **Project:** Team Task Board (P2, pair project) · **Date:** October 25, 2026

HW3 is individual. The repository, Project board, milestones, issues, PRD, and mockup belong to the shared P2 project. My rules file, experiment, and reflection are my own and are on branch `hw3-haomiao`. Development uses Visual Studio Code with GitHub Copilot Chat.

1. Links

Item	Link
Repository	stone-coding/team-task-board-copilot
GitHub Project (board view)	Team Task Board, Sprint Board
Milestones	Sprint 1 and Sprint 2
Issues (13)	Issue list
My HW3 branch (root `AGENTS.md`)	`hw3-haomiao`
`AGENTS.md` (v2)	AGENTS.md
`AGENTS.md` v1 → v2 diff	Compare rules-v1…rules-v2
PRD	docs/prd.md
Mockup (desktop, 375px, HTML source)	docs/mockups
Experiment report	experiment.md
Experiment evidence	docs/hw3/evidence
Run A code (no rules)	`experiment-a-no-rules`
Run B code (`AGENTS.md` v1)	`experiment-b-with-rules`
Evaluation, committed before the runs	`hw3-eval`: hw3/eval

2. Scrum setup

- **Board:** a GitHub Project with Status columns Backlog → Sprint Todo → In Progress → In Review → Done. The "Sprint Board" view uses the board layout grouped by Status, and an `Estimate` field holds story points.
- **Milestones:**
  - Sprint 1: October 26 – November 2, 2026 (due November 2), 6 issues
  - Sprint 2: November 3 – November 10, 2026 (due November 10), 7 issues
- **Issues:** 13, derived from the PRD's user stories (US1–US5) and the P2 technical requirements. Each has:
  - an action-oriented title
  - a user story or technical objective
  - an acceptance-criteria checklist
  - a type label (`feature`, `chore`, `docs`, or `test`) and a priority label
  - a milestone, dependencies, and an estimate
- **Board placement:** Sprint 1 issues (#1–#6) are in Sprint Todo; Sprint 2 issues (#7–#13) are in Backlog.
- **Ownership:** no issue is assigned yet. Ownership will be decided with my partner at Sprint 1 planning.

3. Rules file

`AGENTS.md` is at the repository root of `hw3-haomiao`. GitHub Copilot Chat in Visual Studio Code loads it automatically through the workspace setting `chat.useAgentsMdFile`, and the file asks the assistant to name the applicable rules before coding. Sections:

1. Project overview, with links to the PRD, mockup, and backlog
2. Tech stack with installed versions, plus Next.js 16 specifics (`cacheComponents`, the Suspense pattern, `notFound()` behavior)
3. Architecture, directory structure, and key patterns
4. Naming conventions and coding standards
5. UI components, expected behavior, mockup design tokens, and user flows
6. Testing strategy: red → green → refactor as separate commits, coverage goal, end-to-end policy, definition of done, structure checklist
7. Branch, commit, and pull-request workflow; issue references; backlog map
8. Preferred and avoided libraries
9. Security
10. Accessibility (WCAG 2.1 AA)

11. Rules for AI-assisted implementation, including how Copilot Chat receives the rules
12. Changelog (v1 → v2)

4. Before/after experiment: summary

Two Copilot Chat sessions (Agent mode, same model) started from the same commit with the same prompt, PRD, and mockup:

- **Run A:** no rules.
- **Run B:** `AGENTS.md` v1, loaded automatically.

Run B's first message listed the applicable `AGENTS.md` rules by number, which confirms the rules were loaded. A hidden acceptance suite was committed before either run.

Both runs passed all 10 hidden acceptance tests and the accessibility scan. Run A matched the mockup slightly more closely. The rules changed how the feature was built:

- **Structure:** B used the documented components, Tailwind, shared status labels, and an issue-referencing commit.
- **Framework pattern:** B used the correct Next.js 16 route pattern on the first attempt. A needed a build failure first, then used a workaround limited to known ids.
- **Accessibility:** B kept the column names clean ("To Do"). A's tests locked in a flawed name ("To Do3").
- **Tests:** B added 2 end-to-end tests. A added none.
- **Rules not followed:** B ran its failing tests first but committed once, on the checkout branch, with tests in `tests/unit/`. These gaps drove the v2 revision.

The method, full results, code comparisons, and screenshots follow in the experiment report.

5. Iteration and reflection

- **Iteration:** `AGENTS.md` v1 (tag `rules-v1`) was revised to v2 (tag `rules-v2`). Each change is tied to an observation from run B (experiment report §5; changelog in `AGENTS.md`).
- **Reflection:** included at the end of this document.

HW3 Before/After Experiment: GitHub Copilot Chat with and without `AGENTS.md`

Author: Haomiao Shi (individual HW3). Date: 2026-10-25. Feature: issue **#6, "Build three-column Kanban board with mock data"** (Sprint 1, PRD US4).

1. Method

The same feature was implemented twice in Visual Studio Code with GitHub Copilot Chat. Only the rules differed.

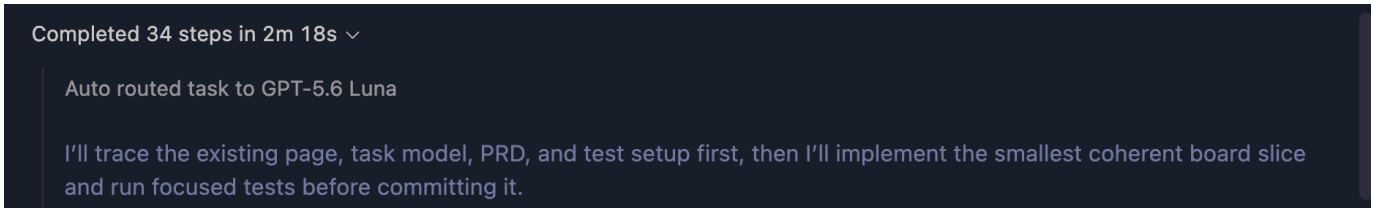
Held constant	Value
Starting code	Tag `experiment-baseline`: scaffold, PRD (`docs/prd.md`), and mockup (`docs/mockups/board.png`). Each run used its own clone. Neither clone could reach the evaluation suite, and the no-rules clone could not reach the rules
Prompt	Identical text pasted into both runs (`evidence/prompt.txt`): a brief feature description that points to the PRD and the mockup
IDE and assistant	Visual Studio Code 1.139.1 with GitHub Copilot Chat in **Agent** mode, in a new chat for each run. Tool and terminal requests were auto-approved for the session. I made no code edits and gave no hints during either run
Model	Copilot "Auto" in both runs. The chat shows that both routed to the same model, GPT-5.6 Luna
Evaluation	Committed before the runs on branch `hw3-eval`: 10 hidden Playwright acceptance tests (`acceptance.spec.ts`) plus a script for lint, types, unit tests, build, and conventions. The tests check only the URL, visible text, and accessible roles. The suite scores 0/10 on the baseline and 10/10 on a reference implementation

Run	Rules
A	No `AGENTS.md`. `.vscode/settings.json` turns off every automatic Copilot instruction source (`chat.useAgentsMdFile: false`)
B	`AGENTS.md` v1, loaded automatically (`chat.useAgentsMdFile: true`). All other settings are the same as A

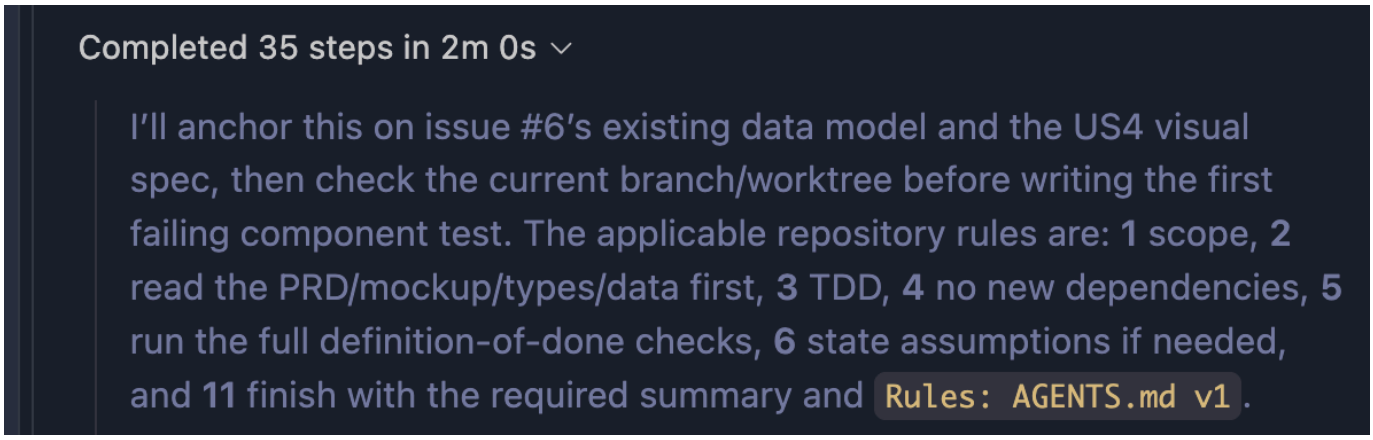
Were the rules actually loaded in run B? Yes:

- Copilot's first message in run B named the applicable rules by their numbers in `AGENTS.md` ("1 scope … 3 TDD … 11 … `Rules: AGENTS.md v1`").
- Its final summary ended with `Rules: AGENTS.md v1`, shown as a linked file reference.
- Neither text appears in run A.

Run A start (no rules)



Run B start (rules loaded)



Deviations and limitations:

- Each condition ran once (n = 1), so the differences are observations, not statistically established effects.
- **Run B's chat started with an extra turn.** It began with an unrelated `pwd` command before the prompt.

- **Run A's agent read Copilot's "memory".** Its contents were not visible, so they may have added context.
- **Formatting edits were excluded.** Both workspaces had small formatting-only edits that were never committed (`run-a` , `run-b`). They were excluded, and scoring used the committed code (A `1252c9d` , B `b31c4c3`).
- **The chat exports were written by Copilot itself, not taken from raw logs.** I asked Copilot to export each chat after the run, and it wrote the files itself because the session log held only a start record. The screenshots are the primary record of what happened.

2. Results

Raw outputs are in `evidence/` .

Metric	A: no rules	B: `AGENTS.md` v1
Design and mockup intent		
Hidden acceptance tests passed (US4 behavior, mockup, accessibility)	**10/10**	**10/10**
Accessibility violations (axe, WCAG 2 A/AA)	0	0
Column region's accessible name	"To Do3", "Done0" (count read as part of the name)	"To Do", "Done" (count badge hidden from the name)
Visual match to the mockup (§4)	Near-identical: reproduced the mockup's CSS	Close: same layout and colors, slightly different spacing, an extra hint line
Code quality and consistency with project patterns		
Production build / lint errors / type errors	pass / 0 / 0	pass / 0 / 0
Next.js 16 dynamic-route rule	**Build failed once**, then used `generateStaticParams` (works only for ids known at build time)	Used the documented `<Suspense>` pattern; no build failure
Styling	149 lines of custom CSS added to `globals.css`	Tailwind utility classes, as the rules require
State logic	Inline in one 90-line route component, with its own label constants	Inline in `Board` , using shared `STATUS_LABELS` from `src/lib/task-status.ts`
New dependencies	0	0
Naming conventions, architecture, and workflow		
Component structure	One default-export `board.tsx` inside the route folder	`src/components/board/Board.tsx` , `BoardColumn.tsx` , `TaskCard.tsx` (named exports, as in the rules)
Commit message	`feat: build mock project kanban board` (no issue reference)	`feat(#6): build three-column kanban board` with `Refs #6`
Feature branch `feature/6-…`	No	**No** (rule not followed)
Test-first workflow	Code first, tests afterwards	Tests written first and run red at 12:10:17, **but committed together with the code in one commit**
Test placement	`tests/unit/board.test.tsx`	`tests/unit/board.test.tsx` (**rule not followed**; the rules ask for tests next to the code)
Quality of generated tests		
New unit/component tests	2	2
New end-to-end tests	0 (ran only the existing smoke test)	2 (board journey; no horizontal scroll at 375px)
Queries by accessible role / by test id	9 / 0	18 / 0
Follow-up corrections and effort		
Human follow-up prompts during the task	0	0
Unplanned failures the agent fixed itself	3: rejected patch, heading-name test failure, build failure	3: `rg` not installed, accessible-name test failure, zsh glob error in `git add`
Steps / time / Copilot credits	34 / 2m 18s / 3.4	35 / 2m 0s / 2.7
Lines changed (files)	+292 −10 (4)	+231 −10 (8)

What the numbers say

1. **The rules did not change whether the feature works.** Both runs pass all 10 hidden tests and the accessibility scan. On visual fidelity, run A without rules was actually closer to the mockup, because it copied the mockup's CSS directly.
2. **The rules changed structure and correctness details:**
   - B split the board into the documented components, used Tailwind and shared status labels, and referenced the issue in its commit.
   - B added two end-to-end tests and avoided the Next.js build failure entirely.
   - B fixed the column's accessible name. A's tests instead accepted the flawed name ("To Do3"), so a screen-reader defect became the expected behavior.
3. **The rules were followed only partly.** B followed the rules it had to apply while writing code. It skipped the ones about git and file placement: it stayed on the checkout branch, made no separate failing-test commit, placed tests in `tests/unit/`, and wrote no pure `src/lib/board.ts` function. These gaps drove the v2 revision (§5).

3. Side-by-side code

Column heading: the count inside the accessible name (A) vs. hidden from it (B)

```
// run A: src/app/projects/[projectId]/board.tsx
<h2 id={`column-${column.status}`}>
  <span>{column.title}</span>
  <span className="task-count">{columnTasks.length}</span>
</h2>
```

```
// run B: src/components/board/BoardColumn.tsx
<h2 id={headingId} className="mb-3 flex items-center justify-between font-semibold">
  {STATUS_LABELS[status]}
  <span aria-hidden="true" className="rounded-full bg-zinc-200 px-2 py-0.5 text-xs font-semibold text-zinc-600">
    {tasks.length}
  </span>
</h2>
```

Tests: asserting the flawed name (A) vs. the intended name (B)

```
// run A: tests/unit/board.test.tsx
expect(screen.getByRole("region", { name: "To Do3" })).toHaveTextContent("3");
```

```
// run B: tests/unit/board.test.tsx
const todoColumn = screen.getByRole("region", { name: "To Do" });
expect(within(todoColumn).getByText("3")).toBeInTheDocument();
```

Route page: prerendering one known id (A) vs. the documented Suspense pattern (B)

```
// run A: src/app/projects/[projectId]/page.tsx (added after the first build failed)
export function generateStaticParams() {
  return [{ projectId: mockProject.id }];
}
```

```
// run B: src/app/projects/[projectId]/page.tsx
export default function ProjectPage({ params }: PageProps<"/projects/[projectId]">) {
  return (
    <Suspense fallback={<main className="p-4">Loading board...</main>}>
      <ProjectBoard params={params} />
    </Suspense>
  );
}
```

Test-first in run B, but not in its history

Run B's chat shows the failing test run before any component existed. The red step is not in git, though: B committed once (`b31c4c3`).

Run B: failing test before implementation

The reference images match the PRD exactly, including the header/breadcrumb and full-width light zinc canvas. I'm adding the focused component tests first; they will intentionally fail because the board components and route do not exist yet, satisfying the repository's red-green workflow before implementation.

Edited board.test.tsx +39 -1

Ran `npm test --` Auto approved for this session (Disable)

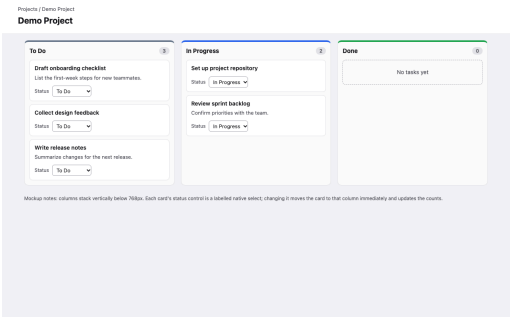
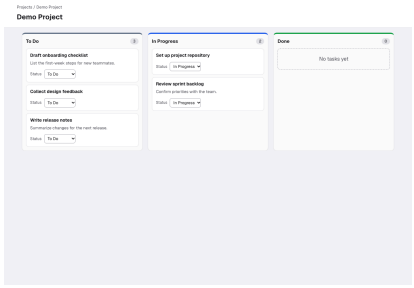
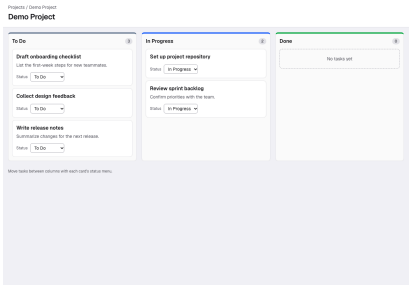
```
npm test -- --run tests/unit/board.test.tsx

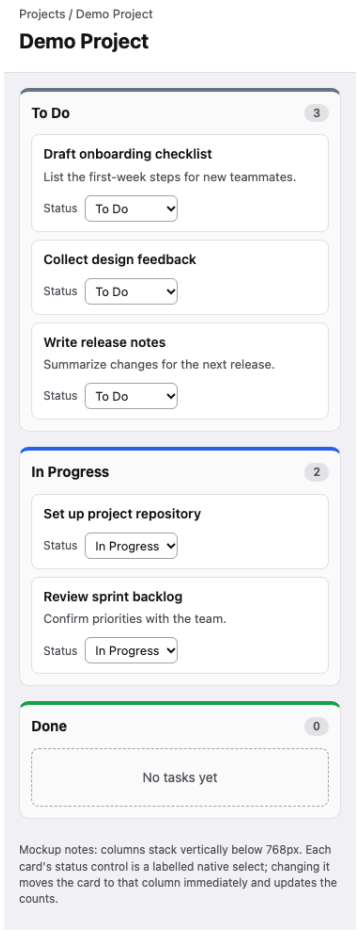
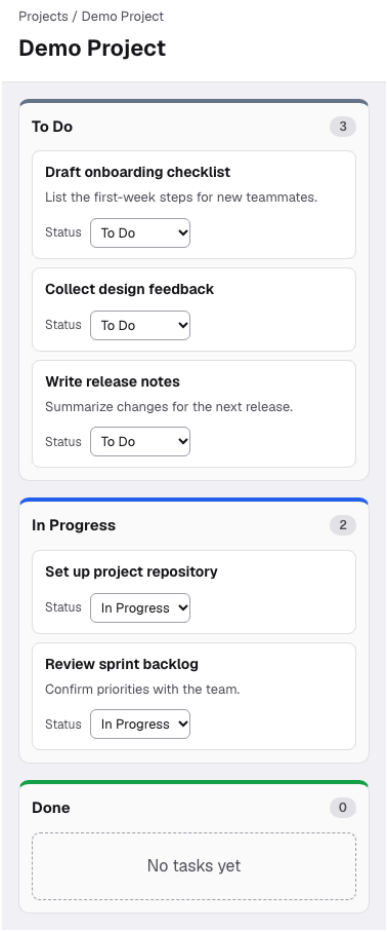
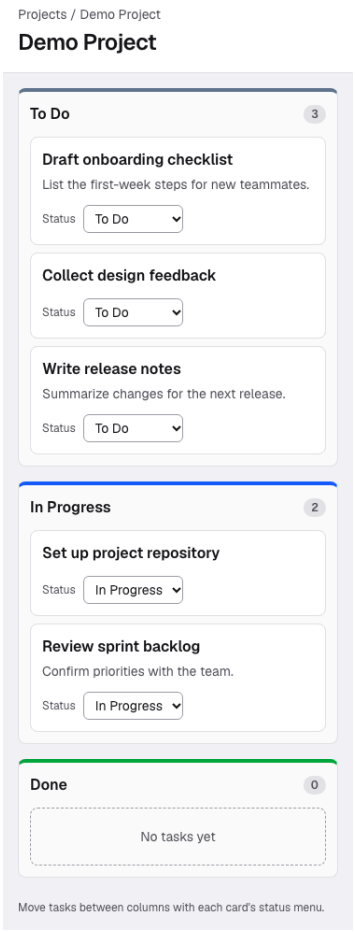
──────────────────────────────────────────────────────────────────────────────
──────────────────[1/1]──

 Test Files  1 failed (1)
      Tests  no tests
   Start at  12:10:17
   Duration  794ms (environment 80%, setup 18%, transform
1%, worker 1%)
```

The red test fails exactly at the missing `Board` module, so the hypothesis holds. I'm implementing the smallest matching slice now: shared ordered status metadata, presentational column/card components, a stateful `Board` client component, and a Suspense-safe dynamic project route that filters the existing demo data and rejects unknown project ids.

4. Screenshots (production build, rendered by the evaluation script)

Mockup	Run A	Run B
		

Mockup	Run A	Run B
		

5. Rules revision v1 → v2

Diff on GitHub

Observation in run B (v1)	v2 change
Wrote tests first but committed everything once, on the checkout branch	§6: create the issue branch first; commit the failing tests alone as `test(#N)` before any implementation; a single-commit feature violates the rule
No pure `src/lib/board.ts`; tests in `tests/unit/`	§6 "Structure checklist" is now part of done: state changes as pure tested functions in `src/lib/`, components in `src/components/<feature>/`, tests next to their files
Its first unplanned test failure came from the count badge inside the heading's accessible name	§5: the heading contains only the column title; the count badge sits beside it
`rg` missing; zsh failed on the `[projectId]` path	§11.8: zsh quoting and the available search tools
Ran the E2E suite but the definition of done did not require it	§6: definition of done adds `npm run test:e2e`
Summary did not list commits	§11.7: the summary lists the branch and commit SHAs, test commit first

Not yet measured: v2 has not been tested in a third Copilot run.

HW3 Reflection: Context Engineering for Team Task Board

Haomiao Shi · CS6983 · October 25, 2026

How the rules file connects the PRD and mockup to code

The PRD defines what the board must do, and the mockup defines how it looks. Neither says how the code should be organized or tested, or what the installed framework version will reject. My `AGENTS.md` fills that gap:

- It maps each mockup element to a component, an HTML element, and an accessible name.
- It records constraints the model cannot infer. For example, Next.js 16 with `cacheComponents` fails the production build when a page reads route parameters outside `<Suspense>`.
- It ties AI output to our workflow: issue branches, `test(#N)` commits before `feat(#N)` commits, and a summary that checks off each acceptance criterion.

In Visual Studio Code, GitHub Copilot Chat loads the file automatically once `chat.useAgentsMdFile` is enabled.

What the experiment showed

I implemented Sprint 1 issue #6 twice with Copilot Chat in Agent mode, from the same commit and prompt, with the same model (GPT-5.6 Luna). Run A had no rules; run B had `AGENTS.md` v1. A hidden acceptance suite was committed before either run.

Both runs passed all 10 hidden tests and the accessibility scan. Run A, without rules, was even closer to the mockup visually, because it copied the mockup's CSS. So the PRD and mockup alone were enough for the visible feature.

The rules changed how the code was built:

- Run B split the board into the documented components, used Tailwind and shared status labels, referenced the issue in its commit, and added two end-to-end tests.
- Run A hit the Next.js build failure and worked around it with `generateStaticParams`, which only works for ids known at build time. Run B used the documented Suspense pattern and never hit the error.
- Run A rendered each column heading so that the screen-reader name became "To Do3", then wrote tests that expect that name. Run B hid the count from the name.

These differences don't change what a user sees today. They decide whether the board can move to real project data in Sprint 2, and whether accessibility defects get caught or locked in by tests.

What did not work, and why I revised the rules

Run B followed the rules that applied while it was writing code, and skipped the ones about process and file placement:

- It wrote failing tests first and ran them red, but committed everything in one commit. The history therefore shows no TDD evidence.
- It stayed on the checkout branch instead of creating `feature/6-…`.
- It kept its tests in `tests/unit/` and its state logic inside the component, although v1 asks for co-located tests and pure functions in `src/lib/`.

My reading is that v1 stated these rules, but nothing in the work forced the agent to act on them.

Version 2 makes them concrete steps instead of guidelines:

- create the branch first
- commit the failing tests on their own, and treat a single-commit feature as a violation
- a structure checklist that is part of the definition of done
- a summary that must list the branch and every commit SHA

It also adds three fixes for problems B hit: where the count badge goes, how to quote paths in zsh, and which search tools are available.

I have not yet run v2, so its effect is a hypothesis, not a measured result.

The limitations are real:

- Each condition ran once.
- Run A's agent read Copilot's memory, whose contents I could not see.
- Run B's chat began with an unrelated `pwd` turn.

How the GitHub Scrum setup supports collaboration

The shared Project board moves work through Backlog, Sprint Todo, In Progress, In Review, and Done. Milestones cover Sprint 1 (October 26 to November 2) and Sprint 2 (November 3 to November 10). Each of the 13 issues has acceptance criteria, labels, an estimate, and dependencies, which makes it a bounded task for an AI assistant:

- the criteria become the first tests;
- the issue number carries into branches, commits, and pull requests;
- the In Review column keeps AI-generated work waiting for a human reviewer.

Issues stay unassigned so that my partner and I can choose ownership together at sprint planning.

What should improve in Sprint 2

- Run the same issue with `AGENTS.md` v2 to check whether the commit and structure rules are now followed.
- Merge my partner's rules and mine into one shared `AGENTS.md` that we both review.
- Add rules for the API layer: input validation, a consistent error format, and owner-scoped queries.
- Enforce the git rules in continuous integration (for example, reject a pull request without a `test(#N)` commit), since the experiment showed that Copilot skipped them even when they were written down.